

Calibration and Inference

Fitting AlgebraicABMs to observed data

Simon Frost

2026-03-30

1 Overview

Statistical calibration bridges the gap between a mechanistic model and real-world observations. AlgebraicABMs provides three inference tools:

Method	Use case	Returns
<code>log_likelihood</code>	Evaluate a single parameter set	Scalar log-likelihood
<code>particle_filter</code>	Sequential marginal likelihood	Log-marginal-likelihood + particle ensemble
<code>abc_reject</code>	Likelihood-free inference	Accepted parameter–distance pairs

This vignette demonstrates all three on a simple birth process where vertices duplicate at a constant rate and we observe noisy population counts.

2 Setup

```
using AlgebraicABMs
using Catlab, AlgebraicRewriting
using Distributions: Exponential, Poisson, logpdf
using Random
Random.seed!(42)

# Import internal helpers for state reconstruction
using AlgebraicABMs.ABMs: _state_at_time, _systematic_resample, Traj
```

3 1. A Simple Birth Process

Our model has a single rule: every vertex duplicates at a constant rate. The schema is `Graph` (vertices + edges), but we only use vertices.

```
# Birth rule:  $\rightarrow \bullet$  (add one vertex)
birth = ABMRule(:birth,
    Rule(id(Graph()),
        create(ob{terminal(Graph)})),
    ContinuousHazard(1.0)) # placeholder rate

println("Birth rule created: each vertex duplicates independently")
```

Birth rule created: each vertex duplicates independently

3.1 Parameterized model

We use `ClosureParams` so the birth rate comes from an exogenous parameter dictionary. This lets the calibration machinery swap in different parameters per particle or sample.

```
# ClosureParams: hazard depends on match + params dict
birth_param = ABMRule(:birth,
    Rule(id(Graph()),
        create(ob(terminal(Graph))))),
    ClosureParams((m, p) -> Exponential(1.0 / p[:rate])))

# Build a parameterized ABM
abm = ABM([birth_param]; params=Dict(:rate => 0.3))

println("Parameterized ABM: birth rate from params[:rate]")
Parameterized ABM: birth rate from params[:rate]
```

4 2. Generating Synthetic Data

Run the “true” model with known parameters and extract noisy observations.

```
# True parameters
true_rate = 0.3
true_abm = ABM([birth_param]; params=Dict(:rate => true_rate))

# Start with 5 vertices
init = Graph(5)

# Run the true simulation
true_traj = run!(true_abm, deepcopy(init);
    maxtime=10.0, maxevent=500, record_history=true)

println("True simulation: $(length(true_traj)) events")
println("  Time span: 0.0 to $(round(true_traj.events[end][1], digits=2))")

True simulation: 3 events
  Time span: 0.0 to 12.39

# Extract observations at regular intervals
obs_times = collect(1.0:1.0:8.0)
true_counts = Int[]

for t in obs_times
    state = _state_at_time(true_traj, t)
    push!(true_counts, nparts(state, :V))
end

println("True population at observation times:")
for (t, c) in zip(obs_times, true_counts)
    println("  t=$(t): $(c) vertices")
end

True population at observation times:
```

```

t=1.0: 5 vertices
t=2.0: 5 vertices
t=3.0: 5 vertices
t=4.0: 6 vertices
t=5.0: 6 vertices
t=6.0: 6 vertices
t=7.0: 7 vertices
t=8.0: 7 vertices

# Add Poisson observation noise
observations = [rand(Poisson(max(c, 1))) for c in true_counts]

println("\nObserved (noisy) counts:")
for (t, obs, truth) in zip(obs_times, observations, true_counts)
    println(" t=$(t): observed=$(obs), true=$(truth)")
end

Observed (noisy) counts:
t=1.0: observed=6, true=5
t=2.0: observed=3, true=5
t=3.0: observed=10, true=5
t=4.0: observed=6, true=6
t=5.0: observed=2, true=6
t=6.0: observed=4, true=6
t=7.0: observed=8, true=7
t=8.0: observed=10, true=7

```

5 3. Log-Likelihood Computation

Given a trajectory, `log_likelihood` evaluates how well it explains the observations. We need:

- `observe_fn(state, t)` — extract the observable from the state
- `log_obs_density(obs, sim)` — log-probability of observing `obs` given simulated value `sim`

```

# Observation function: count vertices in the state
observe_fn(state, t) = nparts(state, :V)

# Observation density: Poisson likelihood
log_obs_density(obs, sim) = logpdf(Poisson(max(sim, 1)), obs)

println("Observation model: Poisson(vertex_count)")
Observation model: Poisson(vertex_count)

```

5.1 Evaluate likelihood at the true parameters

```

ll_true = log_likelihood(true_traj, observations, observe_fn, log_obs_density;
    times=obs_times)

println("Log-likelihood at true parameters: $(round(ll_true, digits=2))")
Log-likelihood at true parameters: -19.53

```

5.2 Likelihood varies with parameters

Run the model at different rates and compare log-likelihoods.

```

test_rates = [0.1, 0.2, 0.3, 0.4, 0.5]
lls = Float64[]

for rate in test_rates
    test_abm = ABM([birth_param]; params=Dict(:rate => rate))
    test_traj = run!(test_abm, deepcopy(init);
        maxtime=10.0, maxevent=500, record_history=true)
    ll = log_likelihood(test_traj, observations, observe_fn, log_obs_density;
        times=obs_times)
    push!(lls, ll)
    println(" rate=$(rate): log-likelihood = $(round(ll, digits=2))")
end

best_idx = argmax(lls)
println("\nBest rate among tested: $(test_rates[best_idx]) (true: $(true_rate))")

rate=0.1: log-likelihood = -20.77
rate=0.2: log-likelihood = -19.38
rate=0.3: log-likelihood = -20.83
rate=0.4: log-likelihood = -19.67
rate=0.5: log-likelihood = -19.84

Best rate among tested: 0.2 (true: 0.3)

```

6 4. Particle Filtering

The bootstrap particle filter estimates the marginal likelihood by running an ensemble of particles forward, weighting them by how well they match each observation, and resampling.

```

result = particle_filter(
    ABM([birth_param]; params=Dict(:rate => true_rate)),
    init,
    observations,
    observe_fn,
    log_obs_density,
    obs_times;
    nparticles=30,
    maxevent=500)

println("Particle filter results:")
println(" Log marginal likelihood: $(round(result.log_marginal_likelihood, digits=2))")
println(" Number of particles: $(length(result.particles))")
println(" Final particle sizes: ", [nparts(p, :V) for p in result.particles])

```

Particle filter results:

Log marginal likelihood: -30.92

Number of particles: 30

Final particle sizes: [15, 15, 16, 16, 16, 16, 17, 17, 16, 16 ... 13, 13, 14, 14, 17, 20, 17, 17, 16,

6.1 Particle diversity

The resampled ensemble reflects the posterior distribution of states. We can examine the spread of population sizes.

```

final_sizes = [nparts(p, :V) for p in result.particles]
println("Final particle population sizes:")

```

```
println("  Min: $(minimum(final_sizes))")
println("  Max: $(maximum(final_sizes))")
println("  Mean: $(round(sum(final_sizes) / length(final_sizes), digits=1))")
println("  Unique particles: $(length(unique(final_sizes)))")
```

Final particle population sizes:

```
Min: 13
Max: 22
Mean: 16.0
Unique particles: 7
```

6.2 Comparing marginal likelihoods across parameters

```
for rate in [0.15, 0.3, 0.5]
  pf = particle_filter(
    ABM([birth_param]; params=Dict(:rate => rate)),
    init,
    observations,
    observe_fn,
    log_obs_density,
    obs_times;
    nparticles=20,
    maxevent=500)
  println("  rate=$(rate): log ML = $(round(pf.log_marginal_likelihood, digits=2))")
end

rate=0.15: log ML = -28.21
rate=0.3: log ML = -35.77
rate=0.5: log ML = -44.13
```

7 5. ABC Rejection Sampling

Approximate Bayesian Computation (ABC) is likelihood-free: it compares summary statistics of simulated and observed data rather than computing exact likelihoods.

We need:

- `summary_fn(traj)` — compute summary statistics from a trajectory
- `distance_fn(sim_summary, obs_summary)` — distance between summaries
- `params_sampler()` — draw parameters from the prior

```
# Summary: final population size
summary_fn(traj) = begin
  if isempty(traj.hist)
    nparts(traj.init, :V)
  else
    nparts(codom(right(traj.hist[end])), :V)
  end
end

# Observed summary: use our last observation as the target
obs_summary = observations[end]

# Distance: absolute difference in final population
distance_fn(sim_summary, obs_summary) = Float64(abs(sim_summary - obs_summary))
```

```

# Prior: uniform over [0.05, 0.8]
params_sampler() = Dict(:rate => 0.05 + 0.75 * rand())

println("ABC setup:")
println("  Summary statistic: final population size")
println("  Observed summary: $(obs_summary)")
println("  Prior: rate ~ Uniform(0.05, 0.8)")

ABC setup:
  Summary statistic: final population size
  Observed summary: 10
  Prior: rate ~ Uniform(0.05, 0.8)

accepted = abc_reject(
  ABM([birth_param]; params=Dict(:rate => 0.1)), # template (params overwritten)
  init,
  obs_summary,
  summary_fn,
  distance_fn,
  nsamples=150,
  threshold=5.0,
  params_sampler=params_sampler,
  maxtime=10.0,
  maxevent=500,
  record_history=true)

println("ABC rejection results:")
println("  Accepted: $(length(accepted)) / 150 samples")

ABC rejection results:
  Accepted: 147 / 150 samples

```

7.1 Analyze accepted parameters

```

if !isempty(accepted)
  accepted_rates = [p[:rate] for (p, _) in accepted]
  accepted_dists = [d for (_, d) in accepted]

  println("Accepted rate statistics:")
  println("  Min: $(round(minimum(accepted_rates), digits=3))")
  println("  Max: $(round(maximum(accepted_rates), digits=3))")
  println("  Mean: $(round(sum(accepted_rates) / length(accepted_rates), digits=3))")
  println("  True rate: $(true_rate)")

  println("\nDistance statistics:")
  println("  Min: $(round(minimum(accepted_dists), digits=2))")
  println("  Max: $(round(maximum(accepted_dists), digits=2))")
  println("  Mean: $(round(sum(accepted_dists) / length(accepted_dists), digits=2))")
else
  println("No samples accepted - try increasing threshold or nsamples")
end

Accepted rate statistics:
  Min: 0.053
  Max: 0.78
  Mean: 0.434

```

True rate: 0.3

Distance statistics:

Min: 0.0

Max: 5.0

Mean: 2.18

8 6. Comparing Approaches

Method	Strengths	Limitations
<code>log_likelihood</code>	Exact (given trajectory), fast for a single evaluation	Requires recorded trajectory history; single-parameter-set evaluation
<code>particle_filter</code>	Handles sequential data; marginal likelihood for model comparison	Computationally expensive; particle degeneracy with many time points
<code>abc_reject</code>	Likelihood-free; works when observation model is intractable	Requires good summary statistics; acceptance rate can be low

Guidelines:

- Use `log_likelihood` for quick parameter scans or gradient-free optimization.
- Use `particle_filter` when you need marginal likelihood estimates for Bayesian model comparison.
- Use `abc_reject` when the observation model is complex or the likelihood is unavailable.

For all methods, keep models small and particle/sample counts modest during development, then scale up for production runs.

9 Summary

```
println("Calibration toolkit summary:")
println("  log_likelihood  - evaluate parameter fit via trajectory")
println("  particle_filter  - sequential Monte Carlo for marginal likelihood")
println("  abc_reject        - likelihood-free approximate Bayesian computation")
println("  _state_at_time    - reconstruct state at any time from trajectory")
```

Calibration toolkit summary:

```
log_likelihood  - evaluate parameter fit via trajectory
particle_filter - sequential Monte Carlo for marginal likelihood
abc_reject      - likelihood-free approximate Bayesian computation
_state_at_time  - reconstruct state at any time from trajectory
```